

Backend Documentation

Core Configuration and State

The system uses configuration variables defined in `config.py` and maintains simple global state in `models.py`.

Global State Variables (`models.py`)

These variables are shared across the application and managed in `utils.py`.

Variable	Type	Description	
<code>CABLE_REMAINING</code>	<code>float</code>	Tracks the remaining length of cable (in meters) the bot has to dispense. Decrementated based on ECI position changes.	
<code>LAST_POSITION_ECI</code>	<code>Tuple[float, float]</code>	The last known Earth Centered Inertial (ECI) 2D position (X, Y) used for calculating cable dispense distance.	
<code>MOVEMENT_THRESHOLD_METERS</code>	<code>float</code>	The Euclidean distance threshold (in Lat/Lon space) used in <code>is_bot_moving_ll</code> to determine if the bot is considered "moving."	
<code>_last_known_location_LL</code>	<code>Dict[str, float]</code>	Stores the previous Latitude/Longitude for motion detection comparison.	

File Paths and Grid Artifacts (`config.py`)

All artifacts, including current values, directions, and grid files, are now configured to use the `.json` format and stored in the `./data` directory by default.

Constant	Default Path	Description
CURRENT_VALUES_PATH	./data/current_values.json	Stores the latest bot status reported via the WebSocket (a JSON representation of the Protobuf status).
DIRECTIONS_PATH	./data/directions.json	Stores high-level operational directions/commands.
WAYPOINTS_PATH	./data/waypoints.json	Stores the target waypoints for path planning.
GRID_DIR	./data	Base directory for all grid-related artifacts.
GRID_IMAGE_PATH	./data/current_image.jpg	The path to the environment map image.
GRID_COORDS_PATH	./data/grid_coordinates.json	The file containing the grid-to-world coordinate mapping.
GRID_OBS_PATH	./data/obstacles.json	The file containing the locations of grid obstacles.
GRID_PATH_PATH	./data/path.json	The file containing the calculated path (list of grid coordinates).

Core Logic and Utilities

The `utils.py` file contains the core computational and file I/O logic.

Motion Detection and Distance Calculation

- `calculate_distance(p1, p2)`: Computes the 2D Euclidean distance between two points $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$. Used specifically for tracking cable length using ECI coordinates.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- `calculate_ll_distance(loc1, loc2)`: Computes the Euclidean distance between two Latitude/Longitude (`LL`) points. **Note**: The code states this is *likely flawed* for real-world distances but is retained for the current implementation.

- `is_bot_moving_ll(current_location_data)` : Uses the `LL` distance function and the `MOVEMENT_THRESHOLD_METERS` to determine if the bot has moved significantly since the last check, updating the global `_last_known_location_LL` .

Core Message Handler (`handle_client_message`)

This function processes incoming raw status data (simulated to be a Protobuf message) from the bot.

1. **Cable Remaining Update**: Calculates the distance traveled using the ECI coordinates from `LAST_POSITION_ECI` to the new reported ECI position, and **decrements** `CABLE_REMAINING` .
2. **Motion Status**: Determines `isMoving` by calling `is_bot_moving_ll` .
3. **Status Object Return**: Returns a complete `circesoft_pb2.CurrentStatus` object with all updated fields.

Path Progress Logic

- `load_path_json(path_json)` : Reads a path from a JSON file as a list of `(col, row)` tuples.
- `path_length(path)` : Calculates the total length of the path in grid units. Diagonal moves count as $\sqrt{2}$ units, and cardinal moves count as 1 unit.
- `closest_path_index(path, pos)` : Finds the index of the path node that is closest to a given position (`pos`) in grid coordinates.
- `distance_along_path(path, idx, pos)` : Calculates the cumulative distance traveled along the path up to a specific index (`idx`), with an optional offset for a position (`pos`) between path nodes.

Networking and Endpoints

The server uses **FastAPI** to manage both REST API endpoints and a persistent **WebSocket** connection.

Application Initialization (`app.py`)

The application:

1. Creates a `FastAPI` app instance.
2. Adds the `CORS Middleware` to allow cross-origin requests from `CORS_ALLOW_ORIGINS` (defaulting to `*`).
3. Includes the `router` from `routes.py` to register all endpoints.
4. Runs using `Uvicorn` on `APP_HOST` and `APP_PORT` (defaulting to `0.0.0.0:8765`).

WebSocket Connections

The system features two WebSocket endpoints:

Endpoint	Purpose	Manager	Logic
<code>/ws</code> (defined by <code>WS_PATH</code>)	Primary Bot/Frontend Communication	N/A	Bidirectional . Receives status updates (Protobuf bytes) and frontend commands (Waypoints/Obstacles/Path JSON) and pushes directions, current values, path, and obstacles to all connected clients by polling file changes every 25ms.
<code>/ws/astarmessages</code>	A* Message Broadcast	<code>ConnectionManager</code>	Clients connect to receive A* path-finding progress messages broadcast via the <code>/send-astar-message</code> REST endpoint.

REST API Endpoints

The endpoints are categorized into General/Data and Grid Artifacts. All `GET` endpoints use **no-cache headers** to ensure clients always get the freshest data.

General Data Endpoints

Method	Path	Function	Description
GET	/health	health()	Simple health check, returns "ok".
GET	/current-values	get_current_values()	Reads and returns <code>current_values.json</code> .
PUT	/current-values	put_current_values()	Overwrites <code>current_values.json</code> with the incoming JSON payload.
GET	/directions	get_directions()	Reads and returns <code>directions.json</code> . Handles a fallback for plain text content.
PUT	/directions	put_directions()	Overwrites <code>directions.json</code> with the incoming JSON payload.
GET	/waypoints	get_waypoints()	Reads and returns <code>waypoints.json</code> .
PUT	/waypoints	put_waypoints()	Overwrites <code>waypoints.json</code> with a validated list of <code>{'r': int, 'c': int}</code> points (and optional 'label').
POST	/send-astar-message	send_astar_message()	Receives a message (e.g., from a path-planning script) and broadcasts it to clients connected to <code>/ws/astarmessages</code> .
GET	/progress	get_path_progress()	Calculates the percentage of the path completed based on the bot's current grid coordinates (X, Y) and the path in <code>PATH_FILE</code> . Note: <code>PATH_FILE</code> is an undefined global in <code>routes.py</code> .

Grid Artifact Endpoints

These endpoints provide access to the files used for visualization and path planning.

Method	Path	Function	Description
GET	/grid/manifest	grid_manifest()	Returns a JSON manifest listing all grid files, their existence, size, modification time, and access path.
GET	/grid/image	get_grid_image()	Returns the <code>current_image.jpg</code> file as

Method	Path	Function	Description
			a <code>FileResponse</code> (media type <code>image/jpeg</code>).
GET	<code>/grid/coordinates</code>	<code>get_grid_coordinates_json()</code>	Reads and returns <code>grid_coordinates.json</code> .
GET	<code>/grid/obstacles</code>	<code>get_grid_obstacles_json()</code>	Reads and returns <code>obstacles.json</code> .
PUT	<code>/grid/obstacles</code>	<code>put_grid_obstacles()</code>	Overwrites <code>obstacles.json</code> with a validated list of <code>{'r': int, 'c': int}</code> points.
GET	<code>/grid/path</code>	<code>get_grid_path_json()</code>	Reads and returns <code>path.json</code> .
PUT	<code>/grid/path</code>	<code>put_grid_path()</code>	Overwrites <code>path.json</code> with a validated list of <code>{'r': int, 'c': int}</code> points.
GET	<code>/grid/bundle</code>	<code>get_grid_bundle()</code>	Streams a ZIP archive containing all grid artifacts (image, coordinates, obstacles, and path).

Grid A* Pathfinder — Full Technical & Design Breakdown

1. High-Level Purpose

This program computes optimal paths in a 31×48 grid using A* with 8-directional movement and feet-based cost metrics. It polls two endpoints (`/waypoints` and `/grid/obstacles`), detects changes via hashing, debounces updates, enforces cooldowns, and publishes the computed polyline path to `/grid/path`.

2. Constants & Globals

- Endpoints: `WAYPOINTS_URL`, `OBSTACLES_URL`, `PATH_PUT_URL`
- Grid bounds: `MAX_ROW=31`, `MAX_COL=48` (Hardcoded for specific image of the TN Tech football field)

- Movement scales: X_SCALE=7.5, Y_SCALE=5.16129, DIAG_SCALE=9.104334
(Hardcoded values for specific grid of TN Tech football field)
- Watch loop timing: POLL_INTERVAL_SEC=2, DEBOUNCE_MS=150,
COOLDOWN_SEC=2

3. Helper Functions

`_unwrap_data`

Normalizes backend payloads to return a clean list.

`_pt_from_rc_or_xy`

Accepts {r,c} or {x,y} formats; converts to (row,col).

`_hash_text`

SHA-256 for change detection.

4. Coordinate Transform System

The simulator uses “internal” coordinates (0=top).

- to_internal flips user r
- from_internal reverses flip

5. Distance System

`segment_length_feet`

Computes diagonal/orthogonal feet distances.

polyline_feet

Sums segment distances across path.

6. A* Algorithm Internals

Implements standard A*:

- g-cost: accumulated feet
- h-cost: diagonal + straight weighted heuristic
- neighbors_8: returns 8 moves
- obstacles excluded
- returns (path, cost)

7. Waypoint Classification

classify_waypoints extracts START, END, and midpoints. Missing either START or END → error.

8. Path Composition

compute_path_through_waypoints:

- Transforms all inputs to internal grid space
- Computes A* legs sequentially: START → midpoints → END
- Merges segments, converts back to user coords
- Validates total feet distance

9. I/O Logic & Endpoint Semantics

fetch_json

GET with robust parsing.

put_path

PUT to /grid/path, prints status + timestamp.

10. Watch Loop & Debounce System

watch_loop:

- Performs initial compute
- Periodically checks for changes
- Hashes waypoints + obstacles
- Debounces (150ms)
- Enforces cooldown (2s)
- Recomputes only when data changes

11. Design Intent for Each Component

- Grid flipping ensures compatibility with backend coordinate conventions
- Hashing prevents unnecessary recomputations
- Cooldown prevents API overuse
- Sequential A* ensures waypoint ordering
- Feet-based metric perfectly matches simulator movement

12. Full Variable Lifecycle Summary

- s,m,e: START, midpoints, END
- obs: obstacle set
- hw, ho: waypoint hash, obstacle hash
- cool_until: timestamp enforcing cooldown
- path, total: computed polyline + feet value